

hmk2/daush2.vhd

```

1  -----
2  --
3  --  SH-2 Data Memory Access Unit
4  --
5  --  This is an implementation of a Data Memory Access Unit for the SH-2 it
6  --  uses the generic memory unit. The generic address unit takes in a number of
7  --  sources and offsets and allows for pre and post increment and decrement.
8  --  The DAU uses this to generate the appropriate address for data accesses.
9  --
10 --  Entities included are:
11 --      ShiftL  - shifter for shifting offset from instruction for multiplication
12 --      DAUSH2  - the data access unit for SH2
13 --
14 --  Revision History:
15 --      12 Apr 26  Simone Shevchuk      Initial revision.
16 --      13 Apr 26  Simone Shevchuk      Set maxincdec bit for generic memory unit
17 --      14 Apr 26  Simone Shevchuk      Added VBR and GBR, and reset. PC masking
18 --      15 Apr 26  Simone Shevchuk      Comments added
19 --      29 Apr 26  Simone Shevchuk      Add zero as an offset option
20 --      01 May 26  Simone Shevchuk      Reset to active low
21 --      12 May 26  Simone Shevchuk      Add load_from_data/data_in for LDC.L GBR/VBR
22 --                                     update data_out mux for MOVA output address_out
23  -----
24
25  --
26  --  ShiftL
27  --
28  --  This is the shifter for doing left shift by 0, 1, or 2 places in the DAU.
29  --
30  --  Generics:
31  --      wordsize - width of the shifter in bits (default 32)
32  --                  must be an even number of bits
33  --
34  --  Inputs:
35  --      S0p      - operand
36  --      ShiftSel - shift to apply to offset, 0 for no shift, 1 for shift by 1 (multiply by 2), 2 for shift by 2
37  --                  (multiply by 4)
38  --
39  --  Outputs:
40  --      SResult - shift result
41  --
42  library ieee;
43  use ieee.std_logic_1164.all;
44
45  entity ShiftL is
46
47      generic (
48          wordsize : integer := 32      -- default width is 8-bits
49      );
50
51      port(
52          S0p      : in    std_logic_vector(wordsize - 1 downto 0);  -- operand
53          ShiftSel  : in    integer range 2 downto 0;                 -- shift to apply to offset
54          SResult   : out   std_logic_vector(wordsize - 1 downto 0)   -- shift result
55      );
56
57  end ShiftL;
58
59
60  architecture dataflow of ShiftL is
61  begin
62
63      -- concatenate the appropriate number of 0s to the right of the operand based on shift sel
64      SResult <= S0p(wordsize - 2 downto 0) & '0'      when ShiftSel = 1 else

```

```

65      SOp(wordsize - 3 downto 0) & "00" when ShiftSel = 2 else
66      SOp(wordsize - 1 downto 0); -- if no shift just pass through
67
68  end dataflow;
69
70
71  --
72  -- DAUSH2
73  --
74  --
75  -- This is a data memory access unit for the SH-2. It uses the generic
76  -- memory access unit to generate the appropriate address for data
77  -- accesses. It outputs along the address bus the address for the next data
78  -- access and updates the base address when an increment or decrement is done
79  -- along the addrsrcout bus. Inputs for the address source can be some register,
80  -- VBR, GBR, or, PC. The offset can be from the instruction or from R0.
81  -- The offset (from instruction) is already assumed to be sign-extended. A shifter
82  -- is also used before the offset is passed in to the generic memory unit to
83  -- to multiply the offset by 1, 2, or 4 depending on the instruction. Only
84  -- post increment and pre decrement are used in the SH-2, and these are set
85  -- accordingly as it can be done by 1 2 or 4 by selecting the incdecbit. The VBR
86  -- and GBR are kept inside the DAU. The VBR is reset to 0 on reset and can be
87  -- loaded with the current PC for interrupt handling. The PC input is masked
88  -- to 0 on the lower 2 bits when used as an address source
89  --
90  --
91  -- Inputs:
92  --   addr_PC   - PC value to use as address source for PC-relative accesses
93  --   addr_Reg  - value from register to use as address source for register indirect accesses
94  --   src_sel   - source to use (log srccnt bits)
95  --   shift_sel - shift (multiplication) to apply to offset, 0 for no shift, 1 for shift by 1 (multiply by 2), 2
96  --   for shift by 2 (multiply by 4)
97  --   offset_ins - offset value given from instruction
98  --   offset_reg - offset value from register for register indirect with offset accesses
99  --   offset_sel - offset to use (log offsetcnt bits) (1 for register offset, 0 for instruction offset)
100  --   inc_dec_sel- whether to increment (0) or decrement (1) address source
101  --   inc_dec_bit- bit of address source to increment/decrement
102  --   pre_post_sel- whether to pre- (0) or post- (1) inc/dec address source, note that
103  --   the generic memory unit so that pre and post are implemented, so that
104  --   pre means use the original source and post means use the incremented/decremented source.
105  --   clock      - the system clock
106  --   reset      - system reset, used to reset VBR to 0
107  --   gbr_write  - signal to load GBR with value from register for interrupt handling
108  --   vbr_write  - signal to load VBR with value from register for interrupt handling
109  --   data_out_sel - signal to select whether to output addr_src_out, address_out, GBR or VBR on data_out for
110  --   instructions that write to memory, used to handle STC instructions that write VBR and GBR to memory
111  --
112  -- Outputs:
113  --   address_out - address bus (wordsize bits)
114  --   addr_src_out - incremented/decremented source, goes to register file to update base address (wordsize bits)
115  --   data_out    - data to write to memory for store instructions (wordsize bits)
116  --
117  --
118  library ieee;
119  use ieee.std_logic_1164.all;
120  use work.array_type_pkg.all;
121
122  entity DAUSH2 is
123  port(
124      addr_PC      : in  std_logic_vector(31 downto 0);
125      addr_reg     : in  std_logic_vector(31 downto 0);
126      src_sel      : in  integer range 3 downto 0;
127      shift_sel    : in  integer range 2 downto 0;
128      offset_ins   : in  std_logic_vector(31 downto 0);
129      offset_reg   : in  std_logic_vector(31 downto 0);
130      offset_sel   : in  integer range 2 downto 0;
131      inc_dec_sel  : in  std_logic;

```

```

130     inc_dec_bit    : in  integer range 31 downto 0;
131     pre_post_sel   : in  std_logic;
132     clock          : in  std_logic;
133     reset          : in  std_logic;
134     gbr_write      : in  std_logic;
135     vbr_write      : in  std_logic;
136     load_from_data : in  std_logic;
137     data_in        : in  std_logic_vector(31 downto 0);
138     data_out_sel    : in  integer range 3 downto 0;
139     address_out     : out std_logic_vector(31 downto 0);
140     addr_src_out    : out std_logic_vector(31 downto 0);
141     data_out       : out std_logic_vector(31 downto 0)
142 );
143
144 end DAUSH2;
145
146 architecture behavioral of DAUSH2 is
147     constant ZERO : std_logic_vector(31 downto 0) := (others => '0'); -- constant for 0 value
148
149     signal temp_PC : std_logic_vector(31 downto 0);
150     -- temp PC with lower 2 bits masked to 0 for word-aligned accesses when using PC as source and shifting by 2 for
word offset
151     signal GBR      : std_logic_vector(31 downto 0); -- global base register for GBR-relative accesses
152     signal VBR      : std_logic_vector(31 downto 0); -- vector base register for VBR-relative accesses,
used to hold vector for interrupt handling
153     signal ShiftRes : std_logic_vector(31 downto 0); -- result from shifter to be used as offset input to
generic memory unit
154     signal src_array: std_logic_array(3 downto 0)(31 downto 0); -- array of address sources to be input into generic
memory unit, includes register, GBR, VBR, and PC
155
156     signal addr_src_out_internal : std_logic_vector(31 downto 0); -- internal signal for addr_src_out before output
muxing for data_out
157     signal address_out_internal : std_logic_vector(31 downto 0); -- internal signal for address_out so it can be
used in data_out mux
158
159     begin
160
161         Regs : entity work.MemUnit
162         generic map (
163             srcCnt      => 4, -- 4 sources, register, VBR, GBR, PC
164             offsetCnt   => 3, -- 2 offsets, from instruction and register
165             maxIncDecBit => 31, -- allow inc/dec at any bit position (byte/word/longword)
166             wordsize    => 32 -- 32-bit wide
167         )
168         port map (
169             AddrSrc      => src_array,
170             SrcSel       => src_sel,
171             AddrOff      => (ZERO, offset_reg, ShiftRes), -- 2 for zero, 1 for register offset, 0 for instruction
offset
172             OffsetSel    => offset_sel,
173             IncDecSel    => inc_dec_sel,
174             IncDecBit    => inc_dec_bit,
175             PrePostSel   => pre_post_sel,
176             Address      => address_out_internal,
177             AddrSrcOut   => addr_src_out_internal
178         );
179
180         -- instantiate the shifter to shift the instruction offset
181         ShiftUnit: entity work.ShiftL
182         generic map (
183             wordsize => 32 -- 32-bit wide
184         )
185         port map (
186             S0p       => offset_ins,
187             ShiftSel   => shift_sel,
188             SResult    => ShiftRes
189         );
190

```

```
191     temp_PC <= addr_PC(31 downto 2) & "00" when src_sel = 3 and shift_sel = 2 else
192         addr_PC; -- if not using PC or register as source, just pass
193
194     -- array of address sources to be input into generic memory unit
195     src_array <= (0 => addr_reg, 1 => GBR, 2 => VBR, 3 => temp_PC);
196
197     address_out <= address_out_internal;
198     addr_src_out <= addr_src_out_internal;
199
200     data_out <= GBR                when data_out_sel = 1 else
201         VBR                      when data_out_sel = 2 else
202         address_out_internal when data_out_sel = 3 else
203         addr_src_out_internal;
204
205     process(clock)
206     begin
207         if rising_edge(clock) then
208             if reset = '0' then
209                 VBR <= (others => '0'); -- reset VBR to 0 on reset
210             else
211                 if vbr_write = '1' then
212                     if load_from_data = '1' then
213                         VBR <= data_in;
214                     else
215                         VBR <= addr_reg;
216                     end if;
217                 end if;
218                 if gbr_write = '1' then
219                     if load_from_data = '1' then
220                         GBR <= data_in;
221                     else
222                         GBR <= addr_reg;
223                     end if;
224                 end if;
225             end if;
226         end if;
227     end process;
228
229
230 end behavioral;
```